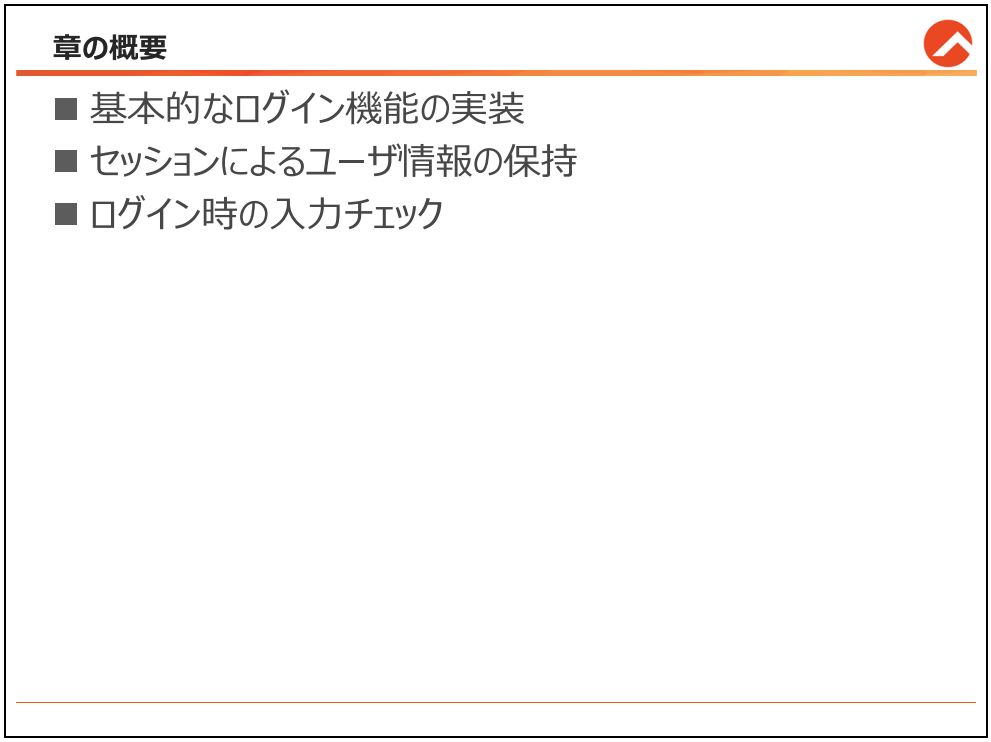
4

# 第4章 セッションと入力チェック

## 章の目標

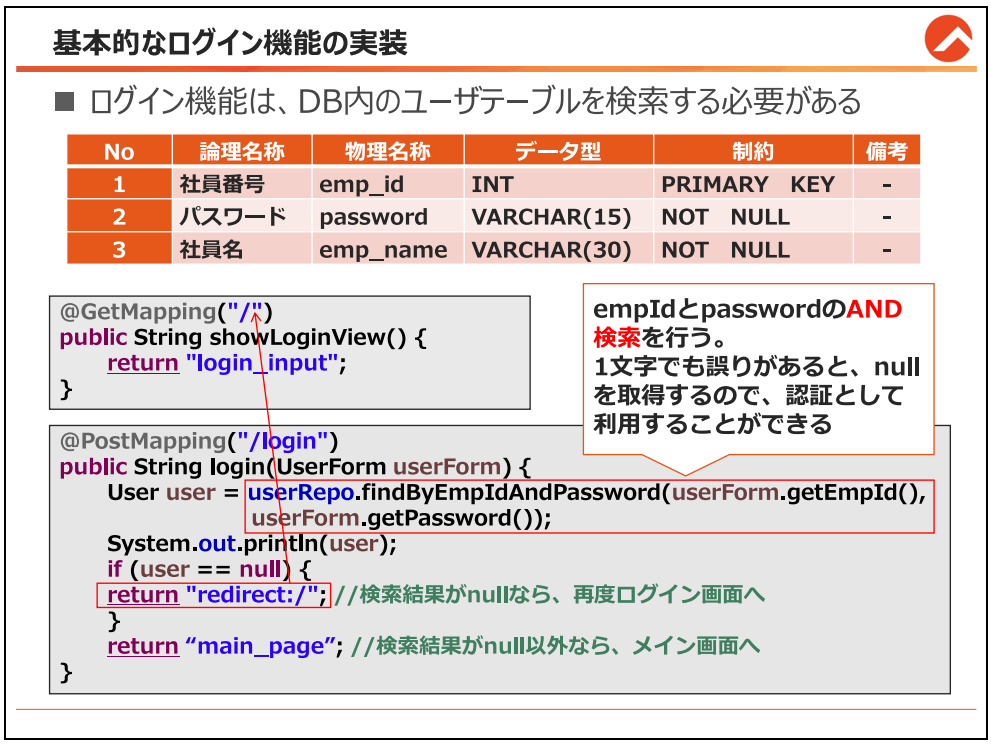
- ■ IDとパスワードによる認証処理を実装できる
- ■ セッションへのデータ保持と、活用方法について理解できる
- ■ 入力チェックとエラーハンドリングを理解し、実装できる

# 章の概要




この章では、ログイン機能を実装する中で、セッションを活用する方法と、様々な入力チェックを簡単にできる方法をご説明します。そして、エラーメッセージの出力を動的に変える方法についてもご紹介します。

## 4.1. 基礎的なログイン機能の実装




今まで学習した内容を踏まえて、ログイン機能を実装してみましょう。ログインには、id とパスワードの値を持っているテーブルが必要となりますので、新たに「ユーザ」テーブルを利用します。テーブルとデータは既にspring_sample データベースに格納済みです。

**【参考】＜User テーブル＞**

| No | 論理名称 | 物理名称 | データ型 | 制約 | 備考 |
|---|---|---|---|---|---|
| 1 | 社員番号 | emp_id | INT | PRIMARY KEY | - |
| 2 | パスワード | password | VARCHAR(15) | NOT NULL | - |
| 3 | 社員名 | emp_name | VARCHAR(30) | NOT NULL | - |

**【参考】＜初期データ＞**

| emp_id | password | emp_name |
|---|---|---|
| 123456 | pass1234 | 山田太郎 |
| 234567 | pass2345 | 佐藤花子 |
| 345678 | pass3456 | 鈴木一郎 |

## 【事前準備①】 エンティティの定義

…/enitity/User.java

```
 1:@Data
 2:@Entity
 3:@Table(name = "user")
 4:public class User {
 5:
 6: @Id
 7:    private Integer empId;
 8:
 9:    @Column(length = 15, nullable = false)
10:    private String password;
11:
12:    @Column(name = "emp_name", length = 30, nullable = false)
13:    private String empName;
14:}
```

**User テーブルを利用しますので、User エンティティも新しく定義する必要があります。**

エンティティの定義については、前の章のProduct とほぼ同じですが、12 行のname 属性を見てください。テーブル内の列名は[emp_name]ですが、Java では通常、変数名の単語ごとの区切りに「アンダースコア(_)」を入れるよりは、大文字と小文字の組み合わせにする、「キャーメルケース」を使います。

**よって、テーブル内の列名と変数名に違いが発生したら、明示的にname 属性をつける必要があります。**

## 【事前準備②】 リポジトリの定義

…/repository/UserRepository.java

```
package jp.co.trainocate.answer.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import jp.co.trainocate.answer.entity.User;

public interface UserRepository extends JpaRepository<User, Integer>{

}
```

エンティティの次は、リポジトリです。3章の定義方法と同様ですので、説明は割愛いたします。

### 【事前準備③】Form クラス定義

…/form/UserForm.java

```
package jp.co.trainocate.answer.form;

import lombok.Data;

@Data
public class UserForm {

    private Integer empId;
    private String password;
    private String empName;
}
```

複数の値をユーザから入力してもらうため、Form クラスを用意する必要があります。
Getter&Setter メソッドや、toString メソッドなどの定義は、lombok ライブラリの@Data をまた利用します。2 章の定義方法と同様ですので、説明は割愛いたします。

### 【作業手順①】 LoginController の作成と、ログイン画面遷移用のメソッド作成

…/controller/LoginController.java

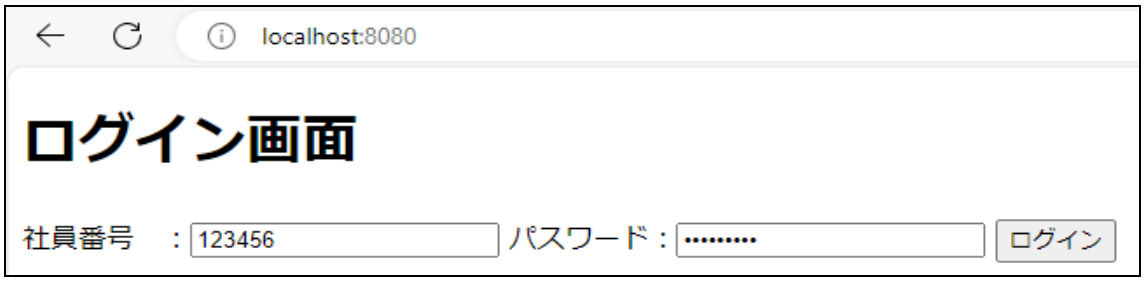
```
@Controller
public class LoginController {
        @Autowired
        private UserRepository userRepo;

        @GetMapping("/")
        public String showLoginView() {
                return "login_input";
        }
```

これから、データベースとやり取りをする予定ですので、user テーブルを操作できるように、UserRepository の依存性の注入をしました。

localhost:8080/ をブラウザに入力すると、ログイン画面へ遷移します。

## 【作業手順②】 ログイン画面の作成




…/resources/templates/login_input.html

```
<body>
<h1>ログイン画面</h1>

<form th:action = "@{/login}" method = "post">
社員番号 ：<input type = "text" name = "empId"/>
パスワード：<input type = "password" name = "password"/>
<input type = "submit" value = "ログイン">
</form>
</body>
```

2 章で作成したログイン画面をそのまま使います。社員番号は、UserForm のフィールド名と合わせるために、[empId]へ修正しました。

## 【作業手順③】 リポジトリにID とパスワードのAnd 検索メソッドの定義

…/repository/UserRepository.java

```
public interface UserRepository extends JpaRepository<User, Integer>{
    public User findByEmpIdAndPassword(Integer empId, String
password);
}
```

login_input.html で送信した**社員ID とパスワードが正しいかどうか判定するため**に、empId とpassword のAnd 検索メソッドを定義します。

主キー検索を含むため、1 行しか取得できないメソッドです。よって、戻り値はエンティティ名のUser にします。

引数として、empId とpassword を入れるように定義します。

## 【作業手順④】 ログイン認証メソッドの定義

…/controller/LoginController.java

```
1:@PostMapping("/login")
2:public String login(UserForm userForm) {
3: User user = userRepo.findByEmpIdAndPassword(userForm.getEmpId(),
                    userForm.getPassword());
4: System.out.println(user);
5: if (user == null) { //結果がnull なら、再度ログイン画面へ
6: return "redirect:/";
7: }
8: return "main_page";
9:}
```

- **3 行目:**

  UserRepository に予め定義しておいたメソッドを実装しました。login_input.html からのリクエストパラメータは、userForm 内に自動格納されているため、getメソッドで取得し、引数に入れました。**検索結果は、最大1 件ですので、User エンティティで受け取ります。**

- **5 行のif 文:**

  **社員ID とパスワードが一致したら、社員データを取得**できますが、入力に誤りがあると、何も取得できないので、user 変数は、null のままになります。このif 文は、**取得できなかった際は、もう一回ログイン画面へ遷移させることで、再入力を促す**コードです。

  5 行のif 文内でreturn されると、即遷移するため、7 行以降が実行されることはありません。

- **8 行目 return “main_page”:**

  データを取得できたなら、ログイン成功ですので、main_page.html へ遷移させます。

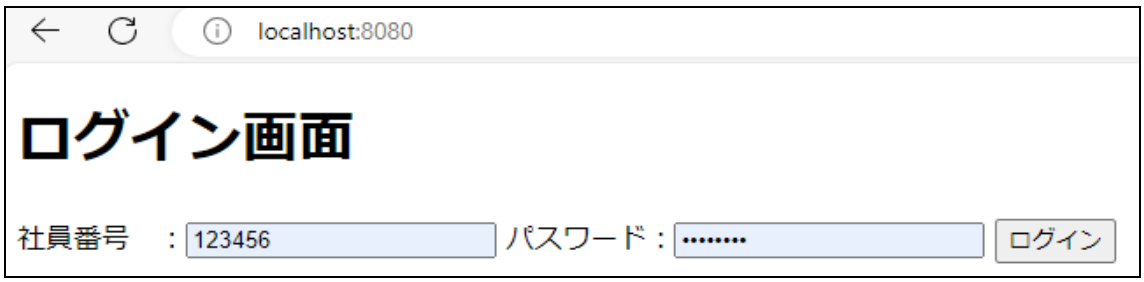
  main_page.html は、下記のように、簡単なhtml ファイルです。

…/resources/templates/main_page.html

```
<!--省略-->
<body>
<h1>メイン画面</h1>
ログインに成功しました。
</body>
```

## 動作確認

http://localhost:8080/

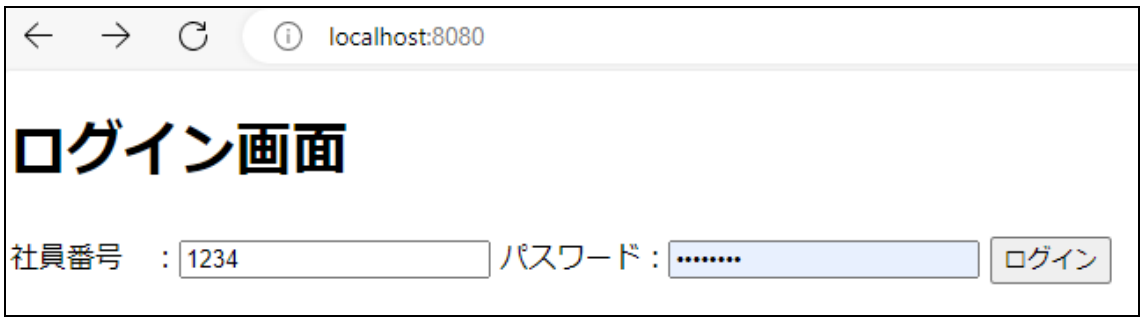



① ログイン画面から、以下のように入力し、[ログイン]ボタンをクリックします。

社員番号:123456パスワード:pass1234

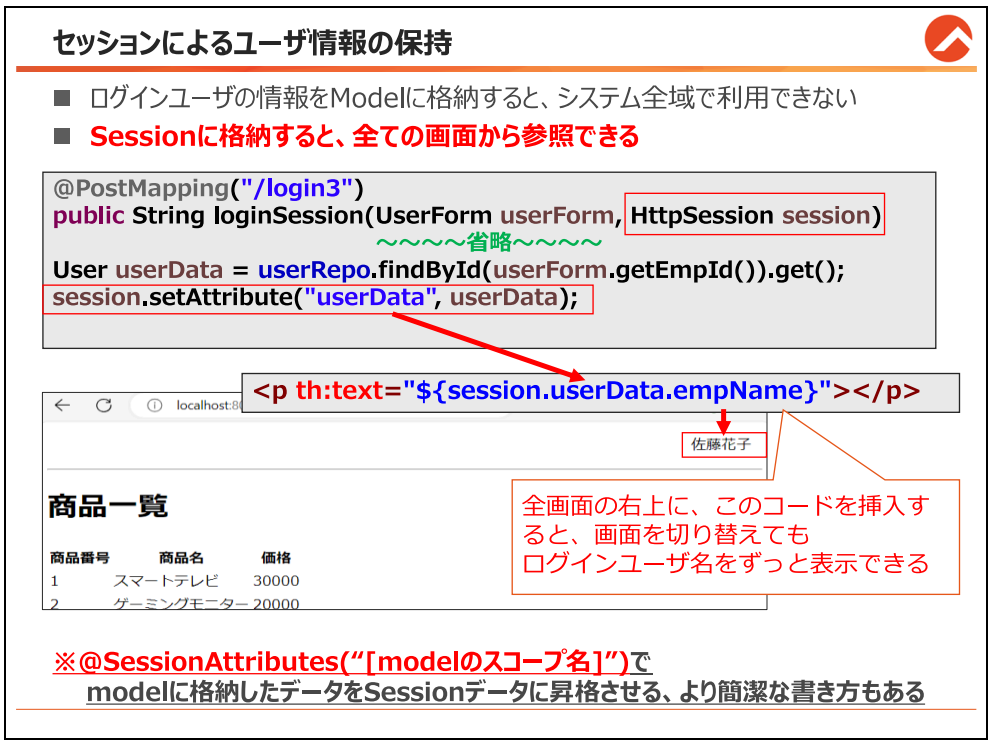



② 正しく入力した場合、メイン画面へ遷移できます。




③ User テーブルに登録されてないId とパスワードの組み合わせで誤ったログインすると、ログイン画面が再度呼ばれることを確認します。

## 4.2. セッションによるユーザ情報の保持



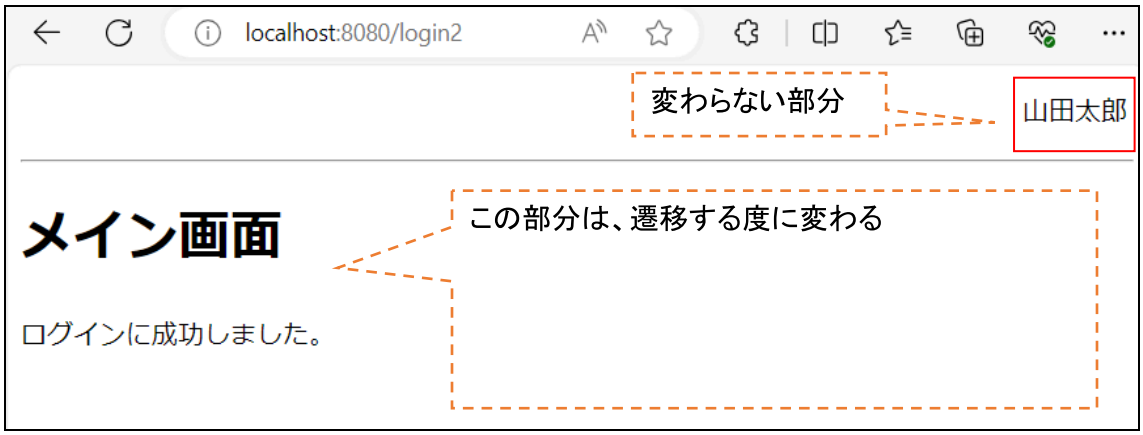

前の節では、一番基本となるログイン機能についてご説明しました。

正しいID とパスワードを入力したら、ログインができるわけですが、ログイン成功後は、システム内の様々な箇所でユーザの情報を必要とします。

そのため、**ログインが成功したタイミングで、そのユーザの情報をセッションに保持させ、システム全体で参照できるようにすることが大事です。**

この節では、セッションにユーザ情報を保持させ、別の画面でそのユーザ情報を参照するところについてご説明します。

### Model に格納した情報は、画面が変わると消える




Web システムでは、多くの異なる画面があり、ユーザはリンクやボタンをクリックすることで、画面遷移を行います。この時、すべての画面で一貫して表示させたいデータもあります。例えば、上の画像のように、ログインしたユーザの名前が各画面上に常に表示される例は、Web システムにおいて一般的なパターンです。

3 章では、コントローラーからビューへデータを送る方法を学びました。ログイン機能を実装する際には、次のようなコードが使用されます。

```
@PostMapping("/login2")
public String loginNormal(UserForm userForm, Model model) {
        User user = userRepo.findByEmpIdAndPasswor
d(userForm.getEmpId(),
userForm.getPassword());
        System.out.println(user);
        if (user == null) {
                return "redirect:/";
        }
        model.addAttribute("userData", user);
        return "main_page";
}
```

このコードは、ユーザが入力した従業員ID（empId）とパスワードが正しい場合、ユーザ情報をModel オブジェクトに格納しているコードです。**Model はビューへデータを送るために使用されますが、さらに画面が変わるとそのデータは保持されません。つまり、ユーザが別の画面に移動すると、先ほどのユーザデータは失われてしまいます。**

Model については、3 章の「findAll メソッドでの全件検索」を参照してください。

## 【作業手順①】 HttpSession 直接利用する方法

Spring でセッションを扱うためには、**HttpSession** のインスタンスを利用します。この方法は、既存のサーブレット&JSP でセッションを利用する方法でもあります。

…/controller/LoginController.java

```
1:@GetMapping("/2")
2:public String showLoginView2() {
3: return "login_input2";
4:}
5:
6:@PostMapping("/login2")
7:public String loginSession(UserForm userForm, HttpSession session) {
8: User user = userRepo.findByEmpIdAndPassword(userForm.getEmpId(),
        userForm.getPassword());
9: System.out.println(user);
10: if (user == null) {
11: return "redirect:/2";
12: }
13: session.setAttribute("userData", user);
14: return "main_page_session";
15:}
```

- **7 行目 (HttpSession session):**

セッションは、HttpSession のインスタンスを利用することで、データをセッションスコープに格納できたり、セッション情報を消したりできます。**Spring のController にURL がマッピングされたメソッドのことを、改めて「ハンドラメソッド」といいます。このハンドラメソッドの引数に HttpSession を指定するだけで、裏でフレームワークが HttpSession のインスタンス生成と、セッション情報の取得を自動で行ってくれます。**

- **13 行目 session.setAttribute("userData", user):**

session.setAttribute メソッドは、model.addAttribute と同じような書き方をします。第一引数には、キーワードを記載して、第二引数には、データを指定します。このコードが実行されると、セッションスコープの中に、"userData"という名前でログインユーザの全情報を保持できるようになります。このデータは、**画面が変わっても、システム全域で呼び出すことができます。**

## 【作業手順②】 セッションスコープ内のデータを参照する方法

セッションスコープ内のデータを参照する方法について、より詳細に解説します。

まず、session.setAttribute メソッドを用いてデータをセッションに格納できます。

格納したセッションデータは、Thymeleaf テンプレートエンジンを介して簡単にアクセスできます。以下はその例です。

…/resources/templates/main_page_session.html

```
<body>
<p th:text="${session.userData.empName}" style="margin-left:90%"></p>
<hr>
<h1>メイン画面</h1>
<p>ログインに成功しました。</p>

</body>
```

赤枠で示された部分のように、session.[キーワード].[変数名]という形式で記述するだけで、セッションスコープのデータを利用できます。この便利さの背景には、Thymeleaf において session が暗黙のオブジェクトとして既に生成されていることがあります。この方法により、HttpSession オブジェクトをどの画面でも直接利用することが可能です。

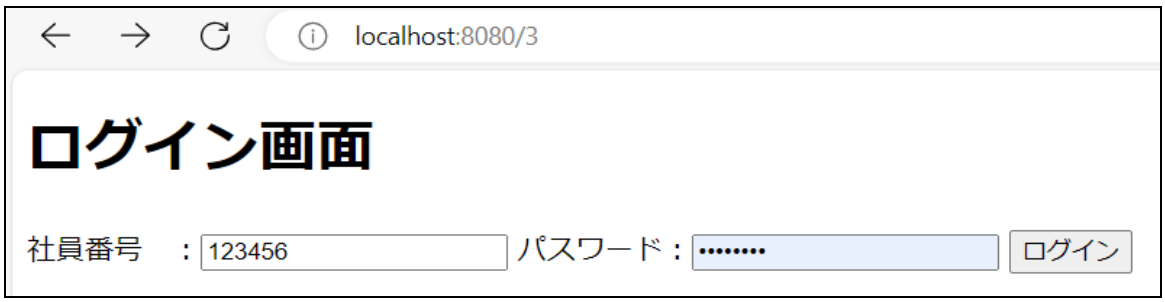
…/resources/templates/product/list.html

```
<body>
 <p th:text="${session.userData.empName}" style="margin-left:90%"></p>
   <hr>
   <h1>商品一覧</h1>
   <table>
        <tr>
        <th>商品番号</th>
        <th>商品名</th>
                    <!--省略-->
```

3 章で記述済みの検索リストを表示するという、ログインとは一切関係ないページ(/product/list.html)にも、同じコードを記述しておきます。

### 動作確認

http://localhost:8080/2 <--こちらのURL へアクセスします。



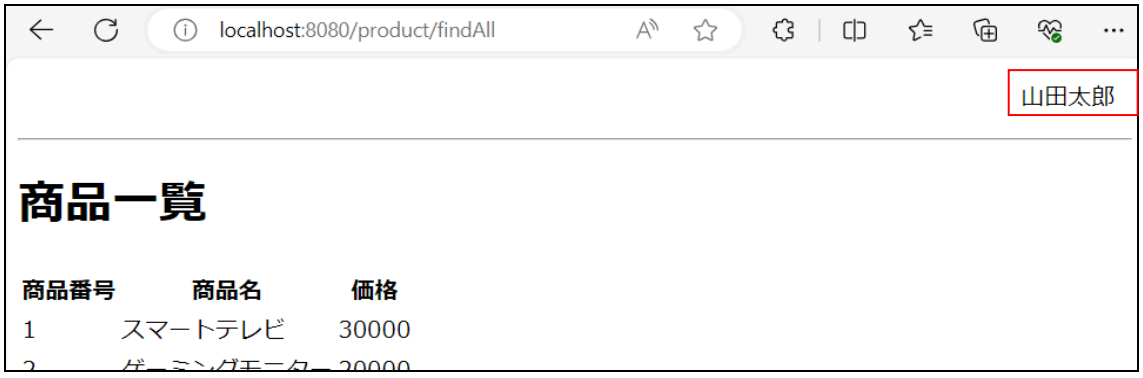

① ログイン画面から、以下のように入力し、[ログイン]ボタンをクリックします。

社員番号:123456パスワード:pass1234




② 画面の右上に、ログインしたユーザ名が表示されていることを確認します。

③ http://localhost:8080/product/findAll <--こちらのURL へアクセスします。




④ 同じコードを書いた、list.html でも、ログインしたユーザ名が確認できます。

### 【参考】 @ SessionAttributes を利用する方法

model に格納したデータと、全く同じデータをセッションにも格納しておきたい場面もあります。その時に活用できる、より簡潔で保守性が高い@SessionAttributes を使う方法についてご紹介します。

…/controller/LoginController.java

```
@Controller
@RequiredArgsConstructor
@SessionAttributes("userData")
public class LoginController {
    private final UserRepository userRepo;
            //省略
    @PostMapping("/login3")
    public String loginNormal(UserForm userForm, Model model) {
          User user = userRepo.findByEmpIdAndPassword(userForm.getEmpId(),
userForm.getPassword());
           System.out.println(user);
           if (user == null) {
                  return "redirect:/3";
           }
           model.addAttribute("userData", user);
           return "main_page_session";
  }
```

上記のソースコードは、model を使ったデータの送信のコードとほぼ同じです。

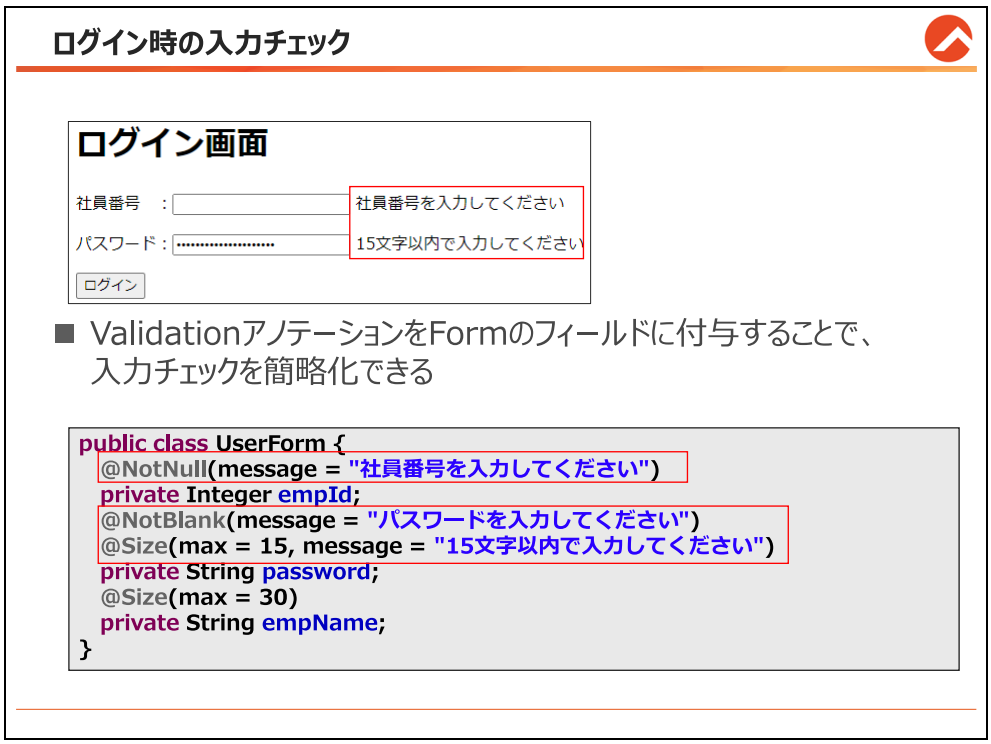
クラス名の上に付与している、**@SessionAttributes("userData")**の部分にご注目ください。

“userData”は、model にuserData を格納する際に使ったキーワードです。@SessionAttributes は、model スコープに格納しているデータを、セッションデータとしても利用できるようにします。

HttpSession を直接使うより、コードが簡潔になり、キー名も統一できますので、こちらの方法を推奨します。

※「**Spring Security API**」を一緒に使うことで、セッションハイジャックなどの脆弱性対策もできます。

## 4.3. ログイン時の入力チェック




Spring では、フォームに入力した値が想定された形式や文字列であるかをチェックする機能を提供しています。その機能のことを「入力チェック（Validation）」と呼びます。この節では Spring のValidation API を活用した、入力チェックの仕方についてご説明いたします。

### 【参考】既存の入力チェックのコード

```
public String validate() {
    if (empId == null) {
        return "社員番号を入力してください";
    }

    if (password == null || password.trim().isEmpty()) {
        return "パスワードを入力してください";
    }

    if (password.length() > 15) {
        return " 15 文字以下で入力してください";
    }

    // 他のフィールドに対するバリデーションも同様に追加

    return null; // 問題がなければnull を返す
}
```

Validation API を使わない、サーブレットなどの場合、入力される値に対して、個別に条件文を作成しておかないといけませんでした。この方法は、記述に時間もかかりますし、保守もしづらい問題がありました。

### 【事前準備】 Validation API を利用する

pom.xml

```
<dependency>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Spring のValidation API をインストールすると、**Form クラスのフィールド名に特定のアノテーションをつけるだけで、入力チェック用のコードを省略できるようになります。**

## 【作業手順①】 Form クラスにValidation のアノテーションを付与

…/form/UserForm.java

```
 1:package jp.co.trainocate.answer.form;
 2:
 3:import jakarta.validation.constraints.NotBlank;
 4:import jakarta.validation.constraints.NotNull;
 5:import jakarta.validation.constraints.Size;
 6:import lombok.Data;
 7:
 8:@Data
 9:public class UserForm {
10:
11: @NotNull(message = "社員番号を入力してください")
12:     private Integer empId;
13:
14: @NotBlank(message = "パスワードを入力してください")
15: @Size(max = 15, message = "15 文字以内で入力してください")
16:     private String password;
17:
18: private String empName;
19:}
```

- **3-5 行目:**

Validationのバージョンによりますが、最新のValidation API は、jakarta.validation.constraints パッケージの方を利用することが推奨されています。

- **10 行目 @NotNull:**

このアノテーションを付与したフィールドに、**入力値がない場合は、入力エラーだと検知**してくれます。**Integerのフィールドは、@NotNull**を付与します。**message属性は、出力するメッセージをカスタマイズ**することができます。

- **14-15 行 @NotBlank、@Size(max = 15):** @NotNull と似ていますが、こちらは**文字列の入力有無チェックに特化**されています。未入力はもちろん、**空文字(“”)まで検知**することができます。@Size は、**文字列の文字数制限**を設定できます。Max で最大文字数、min で最小文字数の設定ができます。

### 【作業手順②】 ログイン入力ページ遷移用のメソッド定義

…/controller/LoginController.java

```
@GetMapping("/4")
        public String showLoginView4(UserForm userForm) {
                return "validation_login_input";
        }
```

入力チェック機能をつける場合は、**必ず入力画面遷移用メソッドの引数にも、Form クラスを入れておく必要**があります。**Form のデータがないと、エラーが出るからです。**よく忘れることが多いので、ご注意ください。

### 【作業手順③】 入力チェック用メソッドの定義

…/controller/LoginController.java

```
1://省略
2:@PostMapping("/login4")
3:public String loginValidation(@Valid UserForm userForm, BindingResult
bdResult) {

4: if (bdResult.hasErrors()) {
5: return "validation_login_input";
6:   }
7: return "main_page";
8:}
```

- **3 行目 @Valid:**

@Valid アノテーションは、対象のオブジェクト（この場合は UserForm）が持つValidation アノテーション（例えば @Size、@NotNull など）に基づいて**入力チェックを実行するために使用**されます。

- **3 行目 BindingResult:**

BindingResult は、@Valid アノテーションによる入力チェックの結果を保持するオブジェクトです。入力チェックエラーの詳細を取得するために使用されます。通常、メソッドの引数として渡されます。**必ず、@Valid をつけた直後(ここでは、第2 引数)に記載しないといけないルールがありますので、ご注意ください。**

- **4 行目 bdResult.hasErrors():**

BindingResult の **hasErrors メソッドは、入力チェック中にエラーが発生したかどうかを確認するために使用されます。エラーが存在する場合は true を、そうでない場合は false を返します。**このメソッドを利用して条件分岐を行い、エラーがある場合は入力フォームに戻り、エラーメッセージをユーザに表示します。

**【作業手順④】 ログイン入力ページの修正**

**…/resources/templates/validation_login_input.html**

```
 1:<body>
 2:<h1>ログイン画面</h1>
 3:
 4:<form th:action="@{/login4}" method="post" th:object="${userForm}">
 5:
 6:<p>社員番号 :<input type = "text" th:field="*{empId}"/>
 7: <span th:errors="*{empId}"></span></p>
 8:
 9:<p>パスワード:<input type="password" th:field="*{password}" />
10: <span th:errors="*{password}"></span></p>
11:
12:<input type="submit" value="ログイン">
13:
14:</form>
15:</body>
```

- **4 行目 th:object="${userForm}":**

th:object 属性は、**「選択対象オブジェクト」**と呼ばれ、フォームのデータをController のオブジェクト（この場合は UserForm オブジェクト）にバインド（結びつけ）するために使用されます。

**formタグに記述することで、フォームの入力フィールドの値が、UserForm オブジェクトの対応するプロパティに自動的に格納されます。つまり、Controller のメソッド内でModel にデータを格納しなくても、Form タグの中で、Thymeleaf 上でデータを受け取れるということです。**

フォーム内では、***{...}**という選択式を使用して、userForm オブジェクトのプロパティに簡潔にアクセスできます。**userForm.empId ではなく、*{empId}だけでアクセスできるように**なります。

- **6 行目 th:field="*{empId}":**

**th:field 属性**は、対応するフィールドの **name, id, および value 属性を自動的に生成**します。ですので、name、id、value を省略できます。

自動生成されるvalue 属性により、フォーム送信後に入力チェックエラーが発生した場合でも、**ユーザが入力した値がフィールドに保持されます**。これにより、ユーザは入力値を再度入力する手間を省くことができます。

なお、*{empId}は、userForm オブジェクトのempId プロパティを参照しています。

- **7 行目 :**

このコードは、empId フィールドに関連する入力チェックエラーがある場合に**エラーメッセージを表示**するために使われます。エラーがない場合は、何も表示されません。

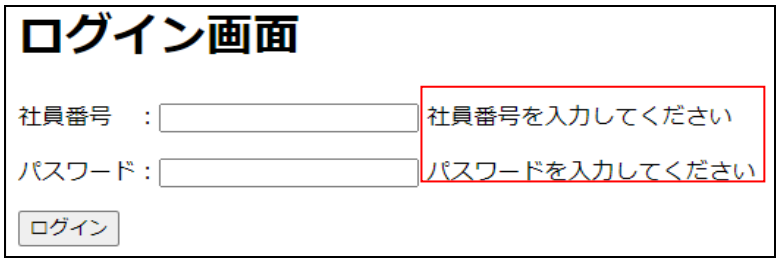
フォームを送信したとき、もし empId フィールドの値が入力チェックのルールに違反していれば（例えば、必須フィールドが空である、など）、 タグの中に入力チェックごとのエラーメッセージが表示されます。

## 動作確認

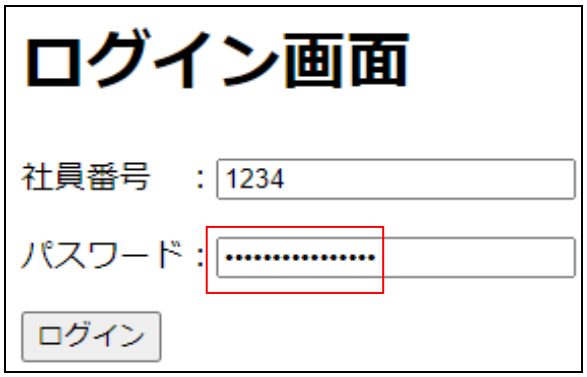
http://localhost:8080/4



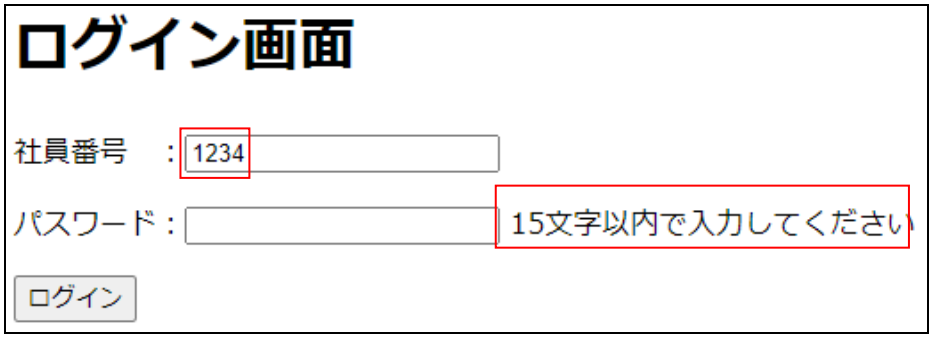

① 何も入力せずに、[ログイン]ボタンをクリックします。



② 入力エラーメッセージが表示されます。




③ パスワードに16 文字以上の文字列を入力し、[ログイン]ボタンをクリックします。




④ 社員番号はそのまま値が入っている状態で、文字数制限エラーが表示されるか確認します。

## 【参考】 ログイン機能の最終版のソースコード

この章では、ログインについて、以下の実装方法についてご説明しました。

① 入力したID とパスワードをUser テーブル内でAnd 検索し、正しいかどうかチェック

② ログインユーザのユーザ情報をセッションに格納し、システム全域で活用

③ Validation アノテーションを使った、入力チェック

では、①②③を全て使った最終版のコードを以下に記述します。

…/resources/templates/login_input_final.html

```
<body>
<h1>ログイン画面</h1>

<form th:action="@{/login5}" method="post" th:object="${userForm}">

<p th:if="${loginError}" th:text="${loginError}"></p> <!--5.1.2 参考-->

<p>社員番号 :<input type = "text" th:field="*{empId}"/>
                    <span th:errors="*{empId}"></span></p>

<p>パスワード:<input type="password" th:field="*{password}" />
                    <span th:errors="*{password}"></span></p>

<input type="submit" value="ログイン">

</form>
</body>
```

…/controller/LoginController.java

```
@GetMapping("/5")
public String showLoginView5(UserForm userForm) {
        return "login_input_final";
}

@PostMapping("/login5")
public String loginFinal(@Valid UserForm userForm, BindingResult bdResult,
HttpSession session, Model model) {
        //入力エラーチェック
        if (bdResult.hasErrors()) {
                return "login_input_final";
        }

        User user = userRepo.findByEmpIdAndPassword(userForm.getEmpId(),
userForm.getPassword());
        //DB 照会後のID&パスワードのチェック
        if (user == null) {
        model.addAttribute("loginError", "社員番号またはパスワードが誤っていま
す。"); // エラーメッセージを追加
        return "login_input_final";
    }

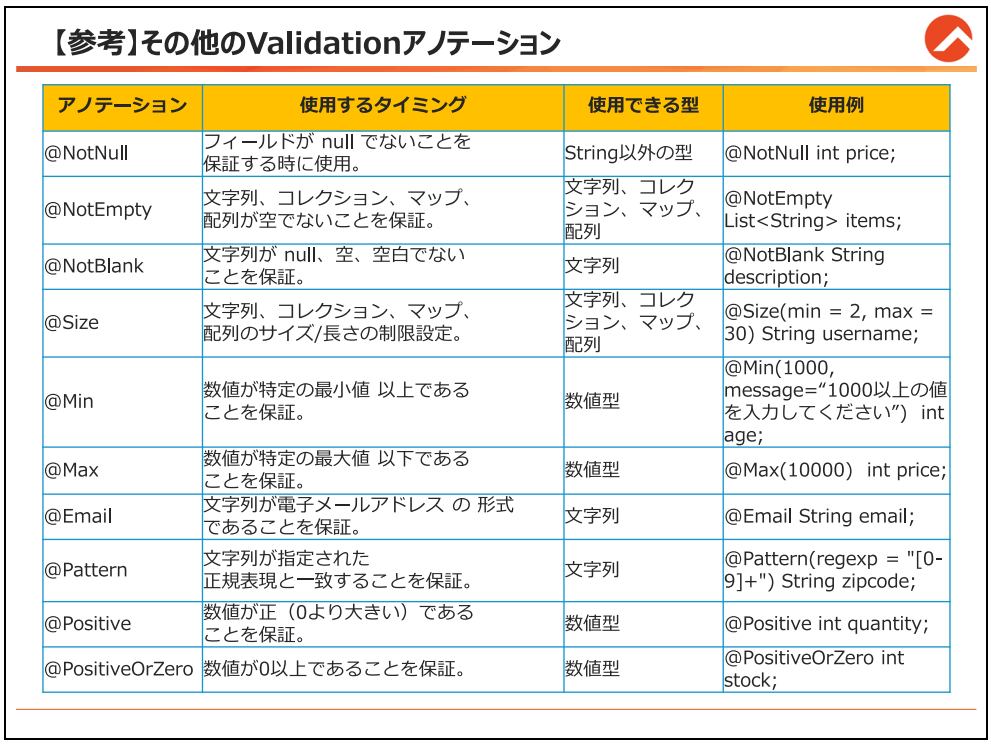
        //正しい入力の場合、ユーザ情報をセッションに格納
        session.setAttribute("userData", user);

        return "main_page_session";
}
```

…/resources/templates/main_page_session.html

```
<body>
<p th:text="${session.userData.empName}" style="margin-left:90%"></p>
<hr>
<h1>メイン画面</h1>
<p>ログインに成功しました。</p>
</body>
```

## 4.3.1.【 参考】その他の Validation アノテーション



【参考】その他のValidationアノテーション

| アノテーション | 使用するタイミング | 使用できる型 | 使用例 |
|---|---|---|---|
| @NotNull | フィールドが null でないことを保証する時に使用。 | String以外の型 | @NotNull int price; |
| @NotEmpty | 文字列、コレクション、マップ、配列が空でないことを保証。 | 文字列、コレクション、マップ、配列 | @NotEmpty List<String> items; |
| @NotBlank | 文字列が null、空、空白でないことを保証。 | 文字列 | @NotBlank String description; |
| @Size | 文字列、コレクション、マップ、配列のサイズ/長さの制限設定。 | 文字列、コレクション、マップ、配列 | @Size(min = 2, max = 30) String username; |
| @Min | 数値が特定の最小値 以上であることを保証。 | 数値型 | @Min(1000, message="1000以上の値を入力してください") int age; |
| @Max | 数値が特定の最大値 以下であることを保証。 | 数値型 | @Max(10000) int price; |
| @Email | 文字列が電子メールアドレス の 形式であることを保証。 | 文字列 | @Email String email; |
| @Pattern | 文字列が指定された正規表現と一致することを保証。 | 文字列 | @Pattern(regexp = "[0-9]+") String zipcode; |
| @Positive | 数値が正（0より大きい）であることを保証。 | 数値型 | @Positive int quantity; |
| @PositiveOrZero | 数値が0以上であることを保証。 | 数値型 | @PositiveOrZero int stock; |

今回の例文で紹介された@NotNull や、@NotBlank、@Size 以外にも、便利なValidation アノテーションが用意されています。この表に書かれているアノテーションだけでも、一般的なWebシステムで使われる入力チェックをケアすることができます。

注意しないといけないのは、上記の表のように、**アノテーションごと「使用できる型」が異なるということです。例えば、数値のデータに@NotBlank を指定するとエラーが出ます。**

@NotNull については、String 型に適用できないわけではありませんが、@NotBlank の方が空文字や空白のケアができるのでより適切です。

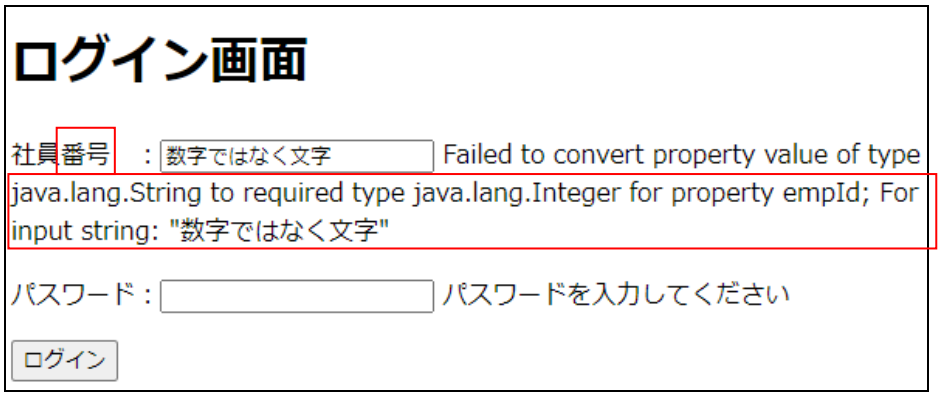
複雑な入力条件は、@Pattern アノテーションで「正規表現」を使うことで、細かく設定できますので、参考にしてください。

## 4.3.2.【 参考】メッセージのカスタマイズ




前の節で、色んな入力チェック用のアノテーションをご説明しましたが、Integer の型に、文字列を入れてしまった時のような、「型の不一致」 や、「日付の整合性チェック」 についてのアノテーションがないことに気づいた方もいらっしゃるでしょう。

**基本的に、型の不一致による入力チェックエラーと、日付関連のエラーは、th:error が自動的にチェックしてくれます。**

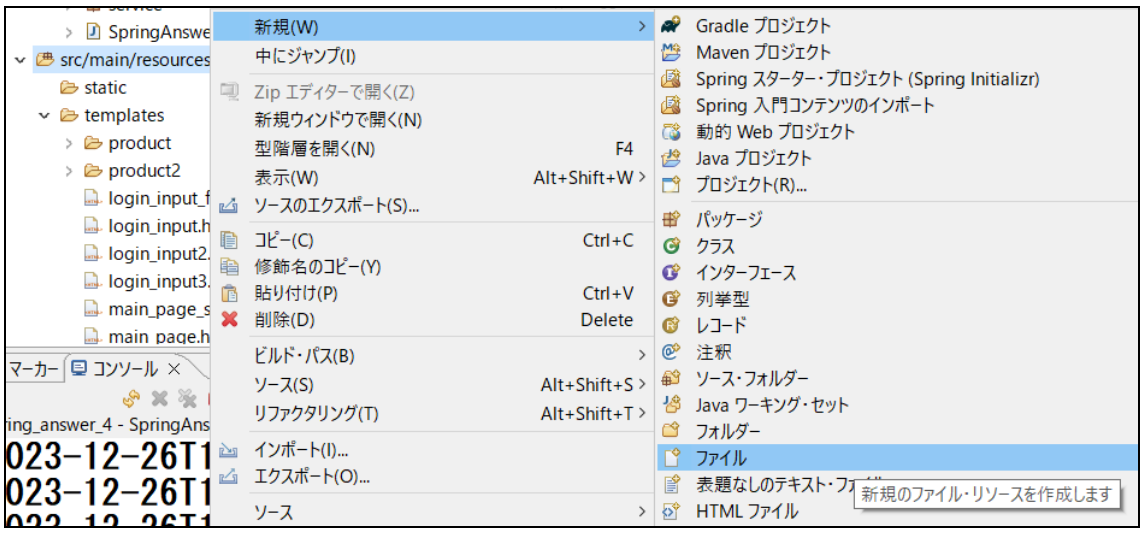



しかし、**自動チェックですので、他のアノテーションのように、message 属性を使った、エラーメッセージの設定ができません。**

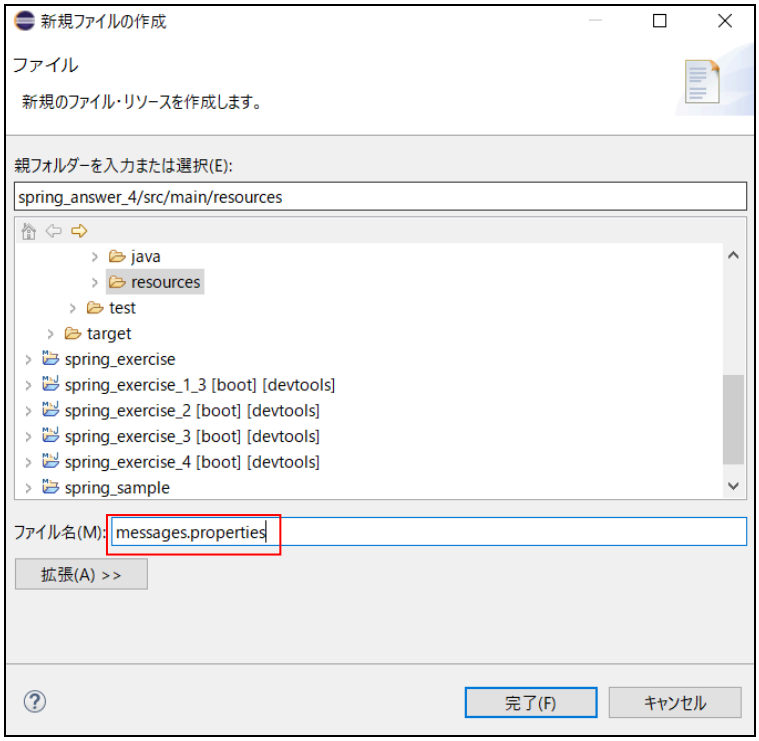
Spring では、メッセージをカスタマイズできるファイルを別途作成できます。

### messages.properties の作成

メッセージプロパティのファイルを活用することで、メッセージのカスタマイズができます。




/src/main/resources に、[ファイル]を新規作成します。




ファイル名は、[messages.properties]にします。

**messages.properties にカスタマイズしたいメッセージを記入**

/resources/messages.properties

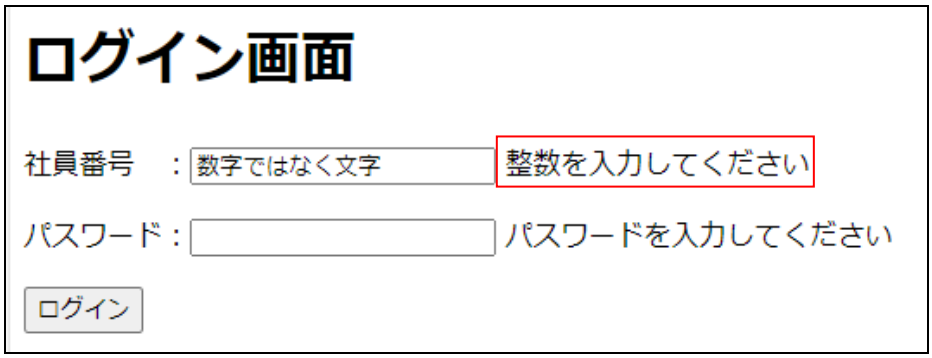
```
typeMismatch.java.lang.Integer=整数を入力してください
typeMismatch.java.lang.Double=小数点を入力してください
typeMismatch.java.util.Date=正しい日付を入力してください
```

messages.properties の中に、エラーメッセージについての表示を変更することができます。

上の例のように、整数と小数点はもちろん、日付についての入力チェックについても、表示されるメッセージを再定義することができます。

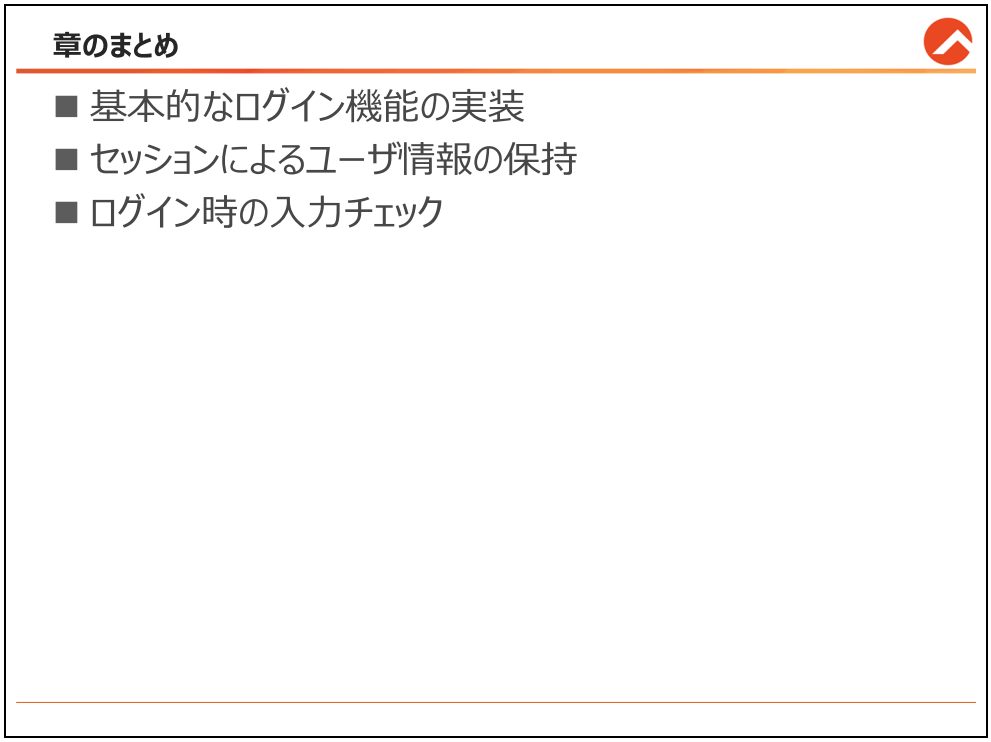
保存後、**[maven install]をすると、修正事項が反映されるようになります。**

外部アプリケーションサーバーで運用するシステムの場合は、必ずサーバーの再起動が必要です。




エラーメッセージが修正されたことを確認できます。

# 章のまとめ




この章では、ログイン機能に必要なセッションや、Validation API の使い方についてご説明しました。